

Data Analysis with Python 2024–2025

Parte 5: Pandas (Parte 3)

Conteúdo Teórico

Tradução e Adaptação: The University of Helsinki MOOC Center

Nota Importante

Este material é uma tradução e adaptação baseada no curso *Data Analysis with Python 2024–2025*, oferecido pela **The University of Helsinki MOOC Center**.

O que você vai aprender nesta página

- Aprender a concatenar datasets.
- Saber como mesclar (merge) DataFrames.
- Saber como dividir um DataFrame em grupos.
- Compreender séries temporais (time series).

Conteúdo

1 Concatenando Datasets

Já vimos na seção sobre NumPy como podemos concatenar arrays ao longo de um eixo: `axis=0` concatena verticalmente e `axis=1` concatena horizontalmente, e assim por diante. Com DataFrames do Pandas, funciona de maneira similar, exceto que os índices das linhas e os nomes das colunas exigem atenção extra. Note também uma pequena diferença no nome: no NumPy é `np.concatenate`, mas no Pandas é `pd.concat`.

Vamos começar considerando a concatenação ao longo do eixo 0 (concatenação vertical). Primeiro, faremos uma função auxiliar para criar facilmente DataFrames para testes.

```
import pandas as pd
import numpy as np

def makedf(cols, ind):
    data = {c: [str(c) + str(i) for i in ind] for c in cols}
    return pd.DataFrame(data, ind)
```

A seguir, vamos criar alguns DataFrames de exemplo:

```
a = makedf("AB", [0, 1])
b = makedf("AB", [2, 3])
c = makedf("CD", [0, 1])
d = makedf("BC", [2, 3])
```

No caso simples a seguir, a função `concat` funciona exatamente como esperaríamos:

```
pd.concat([a, b]) # 0 eixo padrao e 0
```

No entanto, o código a seguir criará índices duplicados:

```
r = pd.concat([a, a])
r.loc[0, "A"] # Retorna duas linhas
```

Isso geralmente não é o que queremos! Existem três soluções para isso. A primeira é proibir a criação de índices duplicados passando o parâmetro `verify_integrity` para a função `concat`:

```
try:
    pd.concat([a, a], verify_integrity=True)
except ValueError as e:
    import sys
    print(e, file=sys.stderr)
```

A segunda opção é solicitar a renumeração automática das linhas:

```
pd.concat([a, a], ignore_index=True)
```

A terceira é usar indexação hierárquica. O Pandas permite múltiplos níveis de índice (embora não entraremos em detalhes profundos aqui).

```
r2 = pd.concat([a, a], keys=['first', 'second'])
r2["A"]["first"][0] # Acesso a um elemento
```

Tudo funciona de forma semelhante quando queremos concatenar horizontalmente:

```
pd.concat([a, c], axis=1)
```

Até agora assumimos que, ao concatenar verticalmente, as colunas de ambos os DataFrames são as mesmas e, ao unir horizontalmente, os índices são os mesmos. Isso, entretanto, não é obrigatório. Quando as colunas não batem, o `concat` as expande com NaNs (um **outer join** ou união):

```
pd.concat([a, d], sort=False)
```

A alternativa é um **inner join** (intersecção), que mantém apenas as colunas em comum:

```
pd.concat([a, d], join="inner")
```

2 Mesclando DataFrames (Merge)

A mesclagem (merge) combina dois DataFrames com base em algum campo comum. Lembremos do DataFrame sobre salários e idades de pessoas:

```
df = pd.DataFrame([[1000, "Jack", 21], [1500, "John", 29]], columns=["Wage", "Name", "Age"])
df2 = pd.DataFrame({"Name": ["John", "Jack"], "Occupation": ["Plumber", "Carpenter"]})
```

A chamada a seguir mesclará os dois DataFrames no campo em comum (**Name**) e, mais importante, manterá os índices alinhados. Isso significa que, mesmo que os nomes estejam em ordens diferentes nas duas tabelas, o **merge** ainda dará o resultado correto.

```
pd.merge(df, df2)
```

Isso foi um exemplo de um merge "um-para-um" simples, onde as chaves na coluna **Name** tinham uma correspondência exata de 1 para 1. Às vezes, nem todas as chaves aparecem em ambos os DataFrames. Por padrão, **pd.merge** faz um **inner join** (mantém apenas as linhas com chaves presentes em ambas as tabelas). Para manter todas as linhas e preencher as faltantes com NaN, usamos **how="outer"**:

```
df3 = pd.concat([df2, pd.DataFrame({"Name": ["James"], "Occupation": ["Painter"]})], ignore_index=True)
pd.merge(df, df3, how="outer")
```

Muitas vezes, relações de "muitos-para-um" ou "muitos-para-muitos" ocorrem nas mesclagens. Nesses casos, todas as combinações de chaves correspondentes são geradas (produto cartesiano para as chaves idênticas).

3 Agregações e Agrupamentos (Groupby)

O Pandas possui uma operação que divide um DataFrame em grupos, realiza alguma operação em cada grupo e, em seguida, combina os resultados em um DataFrame final. Essa funcionalidade de *split-apply-combine* é muito flexível. Inicia-se chamando o método **groupby**.

```
wh = pd.read_csv("https://raw.githubusercontent.com/csmastersUH/data_analysis_with_python_2020/master/kumpula-weather-2017.csv")
wh3 = wh.rename(columns={"m": "Month", "d": "Day", "Precipitation amount (mm)": "Precipitation", "Snow depth (cm)": "Snow", "Air temperature (degC)": "Temperature"})
groups = wh3.groupby("Month")
```

Nada acontece imediatamente (é uma operação "preguiçosa"). O objeto **groupby** sabe como a divisão é feita. Podemos iterar sobre os grupos ou obter um grupo específico usando **groups.get_group(2)**. Para aplicar operações, como a média ou a soma em colunas específicas:

```
groups["Temperature"].mean()
wh3.groupby("Month")["Precipitation"].sum()
```

3.1 Outras formas de operar em grupos

Além das agregações, podemos **filtrar**, **transformar** ou **aplicar** funções genéricas.

- **Filter:** Filtra grupos baseando-se em uma função que retorna um booleano.
- **Transform:** Manipula cada DataFrame de grupo mantendo seu formato original (ex: subtrair a média do mês de cada temperatura diária).
- **Apply:** Muito genérico; requer apenas que a função fornecida retorne um DataFrame, uma Series ou um escalar para o grupo.

4 Séries Temporais (Time Series)

Se uma medição é feita em determinados pontos no tempo, os valores resultantes junto com os tempos de medição compõem uma série temporal. No Pandas, uma Series ou DataFrame cujo índice é composto por datas/horas é uma série temporal.

Podemos converter colunas de Ano, Mês e Dia em um campo `datetime` e defini-lo como o índice:

```
wh2 = wh3.copy()
wh2["Date"] = pd.to_datetime(wh2[["Year", "Month", "Day"]])
wh2 = wh2.set_index("Date")
```

Com um `DatetimeIndex`, podemos facilmente obter fatias (slices) por datas:

```
wh2["2017-01-15":"2017-02-03"]
```

Podemos também criar conjuntos complexos de datas (como "todas as segundas-feiras" ou "apenas dias úteis") usando `pd.date_range`, e usar esses conjuntos para filtrar os dados através do atributo `loc`. O método `rolling` é útil em séries temporais para suavizar gráficos criando médias móveis.

Créditos: Este conteúdo foi originalmente criado pela *The University of Helsinki MOOC Center* para o curso *Data Analysis with Python*.